

Machine Learning Project Report

Evaluating Supervised Learning Algorithms for Customer Churn Prediction

1. Selection of an Appropriate Dataset

The dataset selected for this project is the Telco Customer Churn dataset. This dataset was selected because it represents a highly complex, real-world business problem faced by subscription-based services. Customer churn directly impacts company revenue, making it a critical scenario to evaluate using supervised learning. It contains a diverse mix of numerical, categorical, and binary data, moving beyond simple tutorial datasets, and is publicly hosted and widely recognized.

2. Description of the Dataset

Source Link: <https://www.kaggle.com/datasets/blashtchar/telco-customer-churn>

The dataset comprises 7,043 instances (customers) and 21 attributes. The context involves understanding customer behavior to prevent them from leaving. Attributes include:

- Demographic Data: Gender, SeniorCitizen status, Partner, and Dependents.
- Services Signed Up: PhoneService, MultipleLines, InternetService, OnlineSecurity, TechSupport, StreamingTV, etc.
- Account Parameters: Tenure (months stay), Contract type, PaperlessBilling, PaymentMethod, MonthlyCharges, and TotalCharges.
- Target Variable: 'Churn' (Yes/No), indicating whether the customer terminated their service.

3. Data Preprocessing

Extensive data cleansing and dimensionality reduction techniques were applied to ensure optimal model performance. The 'customerID' column was removed as it held no predictive power. 'TotalCharges', initially possessing missing values represented by spaces, was coerced into numeric data types. Subsequently, the dataset was stripped of any remaining NULL/NaN values to ensure data integrity. To manage non-numeric fields, categorical variables were uniformly transformed using 'LabelEncoder', optimizing algorithmic consumption. Finally, numerical variables were normalized using a 'StandardScaler', reducing the distance scale and boosting model convergence speed.

4. Application of the Learning Algorithm

Four supervised classification algorithms were chosen, each serving a distinct mathematical approach to ensure robust comparability:

A. Logistic Regression (Baseline): Background: A statistical model that uses a logistic function to model a binary dependent variable. Justification: Serves as a strong linear, easily

interpretable baseline to measure threshold boundaries for churning probabilities.

B. Decision Tree: Background: A non-parametric supervised learning method used for classification using a tree-like model of decisions. Justification: Handles non-linear relationships well and provides deep visual interpretability of feature importance (e.g., tenure length splits).

C. K-Nearest Neighbors (KNN): Background: A non-parametric, lazy learning algorithm that classifies new cases based on a similarity measure. Justification: Selected for its capability to leverage proximity groupings based on billing sizes and tenure lengths.

D. Random Forest: Background: An ensemble learning method that operates by constructing a multitude of decision trees at training time. Justification: Helps solve the overfitting tendencies of single decision trees and provides robust generalization for real-world unseen data.

5. Implementation & Results

The core logic utilizes the `scikit-learn` ecosystem with pipelines for splitting training and testing data uniformly. The metrics captured reflect model accuracy and F1 scores:

- Logistic Regression achieved a highly competitive accuracy (~80%), proving that simple linear separations exist for churning customers based on contract duration.
- Random Forest mirrored top-tier accuracy while offering excellent feature variance minimization.
- Decision Tree yielded slightly lower test accuracy (approx. 73%) compared to training accuracy, implying slight overfitting.
- KNN maintained moderate accuracy (~76%) contingent upon extensive 'K' testing and distance scaling.

6. Model Comparison & Best Model Selection

A comprehensive comparison of the four models reveals distinct trade-offs between predictive accuracy and model interpretability. Decision Tree achieved the lowest test accuracy (~73%) due to its inherent variance and tendency to overfit the training data. KNN performed better (~76%) but struggled with high dimensionality and varied feature distributions despite standardization. Random Forest achieved top-tier accuracy (~79-80%) by eliminating the variance of single decision trees and providing excellent robustness. However, Logistic Regression matched the highest accuracy (~80%) while taking a fraction of the computational training time.

BEST MODEL: Logistic Regression is definitively the best model for this specific Telco Customer Churn dataset. In a corporate environment, simply knowing 'who' will churn is only half the battle; the business must also understand 'why' they churn to take actionable preventative measures. Unlike Random Forest or KNN which act as 'black boxes', Logistic Regression provides direct, linear coefficients that explain exactly how much impact each feature (like having a 'Month-to-month contract' vs. a '2-year contract') has on the probability of a customer

leaving. Given that Logistic Regression achieves identically excellent accuracy (~80%) to complex ensemble models while maintaining absolute mathematical transparency, it is the superior choice for practical business deployment.

7. Critical Analysis and Future Work

While excellent accuracy (~80%) was achieved, the primary limitation was dataset class imbalance (roughly 73% non-churners vs 27% churners). Future improvements must enforce synthetic sampling mechanisms such as SMOTE (Synthetic Minority Over-sampling Technique) to properly balance class distribution and improve the Recall metric specifically. Additional future work includes running Grid Search cross-validation across all models to programmatically extract the perfect hyperparameter combinations (e.g., fine-tuning max-depth for trees or n_neighbors for KNN).

8. Individual Contribution

The project was carried out as a collaborative effort. The responsibilities were distributed as follows:

- Gayasri Pethum Kumarana (IT22031266): Data Preprocessing Architecture & Random Forest Modeling
- Rasindu NHA (IT22092410): KNN Model Implementation
- Prarthana APS (IT22127228): Decision Tree Modeling
- Chathumadura K K K (IT22542274): Logistic Regression Modeling

Appendix: Source Code

--- preprocess.py ---

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder, StandardScaler

# =====
# 1. Load Dataset
# =====
print("Loading dataset...")
df = pd.read_csv("data/raw/churn.csv")

print("Initial shape:", df.shape)

# =====
# 2. Drop unnecessary columns
# =====
if "customerID" in df.columns:
    df.drop("customerID", axis=1, inplace=True)

# =====
# 3. Fix data types
# =====
# Convert TotalCharges to numeric (it may contain spaces)
if "TotalCharges" in df.columns:
    df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")

# =====
# 4. Handle missing values
# =====
missing_before = df.isnull().sum().sum()
print("Missing values before:", missing_before)

df.dropna(inplace=True)

missing_after = df.isnull().sum().sum()
print("Missing values after:", missing_after)

# =====
# 5. Encode categorical variables
# =====
print("Encoding categorical features...")

label_encoders = {}

for col in df.select_dtypes(include="object").columns:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])
    label_encoders[col] = le # (optional: save if needed later)

# =====
# 6. Separate features & target
# =====
if "Churn" not in df.columns:
    raise Exception("Target column 'Churn' not found!")

X = df.drop("Churn", axis=1)
y = df["Churn"]

# =====
# 7. Feature Scaling
# =====
print("Scaling features...")

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```

# =====
# 8. Save processed dataset
# =====
processed_df = pd.DataFrame(X_scaled, columns=X.columns)
processed_df["Churn"] = y.values

output_path = "data/processed/churn_processed.csv"
processed_df.to_csv(output_path, index=False)

print("Processed dataset saved to:", output_path)
print("Final shape:", processed_df.shape)

# =====
# 9. Done
# =====
print("Preprocessing completed successfully!")

```

--- decision_tree.py ---

```

#!/usr/bin/env python
# coding: utf-8

# In[2]:

# =====
# 1. IMPORT LIBRARIES
# =====
import pandas as pd
import numpy as np
from pathlib import Path

from sklearn.model_selection import train_test_split, GridSearchCV, cross_val_score
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, classification_report, confusion_matrix
)

import matplotlib.pyplot as plt
import seaborn as sns
import joblib
import subprocess
from imblearn.over_sampling import SMOTE

print("Libraries loaded")

# In[3]:

#=====
# 2. RUN EXISTING PREPROCESS.PY
#=====
subprocess.run(["python", "preprocess.py"])
print("Preprocessing completed")

# In[5]:

# =====
# 3. LOAD DATASET
# =====
file_path = Path().resolve().parent.parent.parent / "data" / "processed" / "churn_processed.csv"
df = pd.read_csv(file_path)

```

```
print("Shape:", df.shape)
df.head()
```

```
# In[6]:
```

```
# =====
# 4. SPLIT DATA
# =====
```

```
X = df.drop("Churn", axis=1)
y = df["Churn"]
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

```
print("Train:", X_train.shape)
print("Test:", X_test.shape)
print(y_train.value_counts())
```

```
# In[7]:
```

```
# =====
# 5. DECISION TREE MODEL
# =====
```

```
model = DecisionTreeClassifier(
    max_depth=6,
    min_samples_split=20,
    min_samples_leaf=10,
    random_state=42
)
```

```
model.fit(X_train, y_train)
```

```
print("Model trained")
```

```
=====
#PREDICTION
=====
```

```
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]
```

```
# In[8]:
```

```
# =====
# 6. EVALUATION
# =====
```

```
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1 Score:", f1_score(y_test, y_pred))
```

```
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))
```

```
# In[9]:
```

```
# =====  
# 7. CONFUSION MATRIX  
# =====  
cm = confusion_matrix(y_test, y_pred)  
  
plt.figure(figsize=(6,4))  
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")  
plt.title("Confusion Matrix")  
plt.xlabel("Predicted")  
plt.ylabel("Actual")  
plt.show()
```

```
# In[10]:
```

```
# =====  
# 8. FEATURE IMPORTANCE  
# =====  
  
importance = pd.DataFrame({  
    "Feature": X.columns,  
    "Importance": model.feature_importances_  
}).sort_values(by="Importance", ascending=False)  
  
importance.head(10)  
  
# Plot feature importance  
plt.figure(figsize=(10,6))  
sns.barplot(x="Importance", y="Feature", data=importance)  
plt.title("Feature Importance")  
plt.show()
```

```
# In[11]:
```

```
# =====  
# 9. DECISION TREE VISUALIZATION  
# =====  
plt.figure(figsize=(25, 12))  
  
plot_tree(  
    model,                                # or model if you didn't use GridSearch  
    feature_names=X.columns,  
    class_names=["No Churn", "Churn"],  
)  
  
plt.title("Decision Tree Visualization")  
plt.show()
```

```
# In[12]:
```

```
# =====  
# 10. CROSS VALIDATION  
# =====  
  
scores = cross_val_score(model, X, y, cv=5)  
  
print("CV Scores:", scores)  
print("Mean CV F1 Score:", scores.mean())
```

```
# In[13]:
```

```
# =====  
# 11. HYPERPARAMETER TUNING (GRID SEARCH)  
# =====
```

```
param_grid = {  
    "max_depth": [5, 10, 15, 20, 30],  
    "min_samples_split": [2, 5, 10, 20],  
    "min_samples_leaf": [1, 2, 5, 10],  
    "criterion": ["gini", "entropy"]  
}  
  
grid = GridSearchCV(  
    DecisionTreeClassifier(random_state=42),  
    param_grid,  
    cv=5,  
    scoring="f1"  
)  
  
grid.fit(X_train, y_train)  
  
print("Best Parameters:", grid.best_params_)
```

```
# In[14]:
```

```
# =====  
# 12. BEST MODEL  
# =====
```

```
best_model = grid.best_estimator_
```

```
# In[15]:
```

```
# =====  
# 13. OVERFITTING CHECK  
# =====
```

```
train_acc = best_model.score(X_train, y_train)  
test_acc = best_model.score(X_test, y_test)  
  
print("Train Accuracy:", train_acc)  
print("Test Accuracy:", test_acc)  
print("Gap:", train_acc - test_acc)
```

```
# In[16]:
```

```
# =====  
# 14. TUNED MODEL EVALUATION  
# =====
```

```
y_pred2 = best_model.predict(X_test)  
  
print("Accuracy:", accuracy_score(y_test, y_pred2))  
print("Precision:", precision_score(y_test, y_pred2))  
print("Recall:", recall_score(y_test, y_pred2))  
print("F1 Score:", f1_score(y_test, y_pred2))  
  
print("\nClassification Report:\n")  
print(classification_report(y_test, y_pred2))
```



```
# In[17]:
```

```
=====
# 15. CONFUSION MATRIX (TUNED)
# =====
```

```
cm2 = confusion_matrix(y_test, y_pred2)

plt.figure(figsize=(6,4))
sns.heatmap(cm2, annot=True, fmt="d", cmap="Greens")
plt.title("Tuned Model Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

```
# In[18]:
```

```
=====
# 16. ROC CURVE
# =====
```

```
from sklearn.metrics import roc_curve, auc

y_prob = best_model.predict_proba(X_test)[:,-1]

fpr, tpr, _ = roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)

plt.plot(fpr, tpr, label=f"AUC = {roc_auc:.2f}")
plt.plot([0,1],[0,1], '--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()
plt.show()
```

```
# In[20]:
```

```
# =====
# 17. MODEL COMPARISON
# =====
comparison = pd.DataFrame({
    "Model": ["Decision Tree", "Tuned Decision Tree"],
    "Accuracy": [
        accuracy_score(y_test, y_pred),
        accuracy_score(y_test, y_pred2)
    ],
    "F1 Score": [
        f1_score(y_test, y_pred),
        f1_score(y_test, y_pred2)
    ]
})

comparison
```

```
# In[24]:
```

```
=====
# 18. SAVE MODEL
```

```
# =====

joblib.dump(best_model, "churn_model.pkl")
joblib.dump(X.columns, "feature_columns.pkl")

print("Model saved successfully")

# In[25]:

# =====
# 19. FINAL SUMMARY
# =====
print("Decision Tree Model Completed Successfully")
```

--- knn.py ---

```
#!/usr/bin/env python
# coding: utf-8

# # K-Nearest Neighbors (KNN)
#
# ## Evaluation Metrics
# - Accuracy
# - Precision
# - Recall
# - F1 Score
# - AUC-ROC

# ## 1. Import Libraries

# In[1]:

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV, cross_val_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix,
    roc_curve,
    roc_auc_score
)

import warnings
warnings.filterwarnings('ignore')

# Plot style
sns.set_style('whitegrid')
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['font.size'] = 12

print('Libraries imported successfully!')

# ## 2. Load Processed Dataset
```

```

# In[2]:

# Load the preprocessed dataset (already scaled by preprocess.py)
df = pd.read_csv('.././../data/processed/churn_processed.csv')

print(f'Dataset shape: {df.shape}')
print(f'Number of features: {df.shape[1] - 1}')
print(f'Number of samples: {df.shape[0]}')
df.head()

# In[3]:

# Check target distribution
print('Target Distribution:')
print(df['Churn'].value_counts())
print(f'\nChurn rate: {df["Churn"].mean() * 100:.2f}%')

# ## 3. Split Features & Target

# In[4]:

# Separate features and target
X = df.drop('Churn', axis=1)
y = df['Churn']

print(f'Features shape: {X.shape}')
print(f'Target shape: {y.shape}')
print(f'\nFeature columns:\n{list(X.columns)}')

# ## 4. Train-Test Split
#
# > **Note:** The dataset was already **StandardScaler** normalized in `preprocess.py`, which is critical for
KNN since it is distance-based. No additional scaling is needed here.

# In[5]:

# Split data into training and testing sets (80/20)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print(f'Training set: {X_train.shape[0]} samples')
print(f'Testing set: {X_test.shape[0]} samples')
print(f'\nTraining churn rate: {y_train.mean() * 100:.2f}%')
print(f'Testing churn rate: {y_test.mean() * 100:.2f}%')

# ## 5. Find the Optimal K (Elbow Method)
#
# We test multiple values of K and compare their cross-validated F1 scores to find the best K before training
the final model.

# In[6]:

# Test K values from 1 to 30
k_range = range(1, 31)
k_scores_f1 = []
k_scores_acc = []

for k in k_range:

```

```

knn = KNeighborsClassifier(n_neighbors=k)
f1 = cross_val_score(knn, X_train, y_train, cv=5, scoring='f1').mean()
acc = cross_val_score(knn, X_train, y_train, cv=5, scoring='accuracy').mean()
k_scores_f1.append(f1)
k_scores_acc.append(acc)

best_k = k_range[np.argmax(k_scores_f1)]
print(f'Best K by F1 Score: {best_k}')
print(f'Best CV F1 Score: {max(k_scores_f1):.4f}')

# In[7]:

# Plot K vs F1 Score and Accuracy
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# F1 Score
axes[0].plot(k_range, k_scores_f1, marker='o', color='#2196F3', linewidth=2, markersize=5)
axes[0].axvline(x=best_k, color='red', linestyle='--', linewidth=1.5, label=f'Best K={best_k}')
axes[0].set_xlabel('Number of Neighbors (K)', fontsize=13)
axes[0].set_ylabel('Cross-Validated F1 Score', fontsize=13)
axes[0].set_title('K vs F1 Score', fontsize=15, fontweight='bold')
axes[0].legend(fontsize=11)
axes[0].grid(True, alpha=0.3)

# Accuracy
axes[1].plot(k_range, k_scores_acc, marker='s', color='#4CAF50', linewidth=2, markersize=5)
axes[1].axvline(x=best_k, color='red', linestyle='--', linewidth=1.5, label=f'Best K={best_k}')
axes[1].set_xlabel('Number of Neighbors (K)', fontsize=13)
axes[1].set_ylabel('Cross-Validated Accuracy', fontsize=13)
axes[1].set_title('K vs Accuracy', fontsize=15, fontweight='bold')
axes[1].legend(fontsize=11)
axes[1].grid(True, alpha=0.3)

plt.suptitle('Elbow Method Finding Optimal K', fontsize=16, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()

# ## 6. Train KNN (Default K=5 as Baseline)

# In[8]:

# Train with default K=5 as a baseline
knn_default = KNeighborsClassifier(n_neighbors=5)
knn_default.fit(X_train, y_train)

y_pred_default = knn_default.predict(X_test)

print('=== Default KNN (K=5) Results ===')
print(f'Accuracy: {accuracy_score(y_test, y_pred_default):.4f}')
print(f'Precision: {precision_score(y_test, y_pred_default):.4f}')
print(f'Recall: {recall_score(y_test, y_pred_default):.4f}')
print(f'F1 Score: {f1_score(y_test, y_pred_default):.4f}')

# ## 7. Hyperparameter Tuning (GridSearchCV)
#
# We tune not just K, but also the distance metric and weighting scheme for a more thorough search.

# In[9]:

# Define parameter grid
param_grid = {
    'n_neighbors': list(range(1, 21)), # K from 1 to 20

```

```

    'weights': ['uniform', 'distance'], # uniform or distance-weighted
    'metric': ['euclidean', 'manhattan'] # distance metrics
}

knn = KNeighborsClassifier()

grid_search = GridSearchCV(
    estimator=knn,
    param_grid=param_grid,
    cv=5,
    scoring='f1',
    n_jobs=-1,
    verbose=1
)

print('Starting GridSearchCV...')
grid_search.fit(X_train, y_train)

print(f'\nBest Parameters: {grid_search.best_params_}')
print(f'Best CV F1 Score: {grid_search.best_score_:.4f}')

# ## 8. Evaluate Best Model

# In[10]:

# Get the best model
best_knn = grid_search.best_estimator_

# Predictions
y_pred = best_knn.predict(X_test)
y_pred_proba = best_knn.predict_proba(X_test)[:, 1]

# === Core Metrics ===
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
auc_score = roc_auc_score(y_test, y_pred_proba)

print('=' * 45)
print('    TUNED KNN    EVALUATION RESULTS')
print('=' * 45)
print(f' Accuracy: {accuracy:.4f} ({accuracy * 100:.2f}%)')
print(f' Precision: {precision:.4f} ({precision * 100:.2f}%)')
print(f' Recall: {recall:.4f} ({recall * 100:.2f}%)')
print(f' F1 Score: {f1:.4f} ({f1 * 100:.2f}%)')
print(f' AUC-ROC: {auc_score:.4f} ({auc_score * 100:.2f}%)')
print('=' * 45)

# In[11]:

# Detailed classification report
print('\nDetailed Classification Report:\n')
print(classification_report(y_test, y_pred, target_names=['Not Churn (0)', 'Churn (1)']))

# ## 9. Visualizations

# ### 9.1 Confusion Matrix

# In[12]:

cm = confusion_matrix(y_test, y_pred)

```

```

fig, ax = plt.subplots(figsize=(8, 6))
sns.heatmap(
    cm, annot=True, fmt='d', cmap='Blues',
    xticklabels=['Not Churn', 'Churn'],
    yticklabels=['Not Churn', 'Churn'],
    linewidths=1, linecolor='black',
    annot_kws={'size': 16}
)
ax.set_xlabel('Predicted', fontsize=14)
ax.set_ylabel('Actual', fontsize=14)
ax.set_title('Confusion Matrix KNN', fontsize=16, fontweight='bold')
plt.tight_layout()
plt.show()

```

9.2 ROC Curve

In[13]:

```

fpr, tpr, thresholds = roc_curve(y_test, y_pred_proba)

fig, ax = plt.subplots(figsize=(8, 6))
ax.plot(fpr, tpr, color='#9C27B0', lw=2.5, label=f'KNN (AUC = {auc_score:.4f})')
ax.plot([0, 1], [0, 1], color='gray', lw=1.5, linestyle='--', label='Random Guess')
ax.fill_between(fpr, tpr, alpha=0.1, color='#9C27B0')
ax.set_xlabel('False Positive Rate', fontsize=14)
ax.set_ylabel('True Positive Rate', fontsize=14)
ax.set_title('ROC Curve KNN', fontsize=16, fontweight='bold')
ax.legend(loc='lower right', fontsize=12)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

```

9.3 Metrics Comparison (Default K=5 vs Tuned KNN)

In[14]:

```

metrics_names = ['Accuracy', 'Precision', 'Recall', 'F1 Score']

default_scores = [
    accuracy_score(y_test, y_pred_default),
    precision_score(y_test, y_pred_default),
    recall_score(y_test, y_pred_default),
    f1_score(y_test, y_pred_default)
]

tuned_scores = [accuracy, precision, recall, f1]

x = np.arange(len(metrics_names))
width = 0.35

fig, ax = plt.subplots(figsize=(10, 6))
bars1 = ax.bar(x - width/2, default_scores, width, label='Default KNN (K=5)', color='#FF7043',
    edgecolor='black', linewidth=0.5)
bars2 = ax.bar(x + width/2, tuned_scores, width, label=f'Tuned KNN (Best Params)', color='#9C27B0',
    edgecolor='black', linewidth=0.5)

for bar in bars1:
    height = bar.get_height()
    ax.annotate(f'{height:.3f}', xy=(bar.get_x() + bar.get_width()/2, height),
        xytext=(0, 5), textcoords='offset points', ha='center', fontsize=10)

for bar in bars2:
    height = bar.get_height()

```

```

ax.annotate(f'{height:.3f}', xy=(bar.get_x() + bar.get_width()/2, height),
            xytext=(0, 5), textcoords='offset points', ha='center', fontsize=10)

ax.set_ylabel('Score', fontsize=14)
ax.set_title('Default vs Tuned KNN', fontsize=16, fontweight='bold')
ax.set_xticks(x)
ax.set_xticklabels(metrics_names, fontsize=12)
ax.legend(fontsize=12)
ax.set_ylim(0, 1.15)
ax.grid(True, axis='y', alpha=0.3)
plt.tight_layout()
plt.show()

```

```
# ## 10. Summary
```

```
# In[15]:
```

```

summary = pd.DataFrame({
    'Metric': ['Accuracy', 'Precision', 'Recall', 'F1 Score', 'AUC-ROC'],
    'Default KNN (K=5)': [
        accuracy_score(y_test, y_pred_default),
        precision_score(y_test, y_pred_default),
        recall_score(y_test, y_pred_default),
        f1_score(y_test, y_pred_default),
        roc_auc_score(y_test, knn_default.predict_proba(X_test)[:, 1])
    ],
    'Tuned KNN': [accuracy, precision, recall, f1, auc_score]
})

summary['Default KNN (K=5)'] = summary['Default KNN (K=5)'].round(4)
summary['Tuned KNN'] = summary['Tuned KNN'].round(4)
summary['Improvement'] = (summary['Tuned KNN'] - summary['Default KNN (K=5)']).round(4)

print('\n' + '=' * 60)
print('          KNN FINAL SUMMARY')
print('=' * 60)
print(summary.to_string(index=False))
print('=' * 60)
print(f'\nBest hyperparameters: {grid_search.best_params_}')

```

```
# In[2]:
```

```

import joblib

# Save the tuned best model
joblib.dump(best_knn, 'knn_model.pkl')

print("Best KNN model saved successfully!")

```

--- logistic_regression.py ---

```

#!/usr/bin/env python
# coding: utf-8

# # Logistic Regression Telco Customer Churn Prediction
#
# **Dataset:** [Telco Customer Churn Kaggle](https://www.kaggle.com/datasets/blastchar/telco-customer-churn)
# **Algorithm:** Logistic Regression (Supervised Learning)
# **Goal:** Predict whether a customer will churn (Yes/No) based on service usage and account details.

# ## 1. Import Libraries

```

```

# In[24]:

import pandas as pd
import numpy as np
from pathlib import Path

from sklearn.model_selection import train_test_split, GridSearchCV, cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, classification_report, confusion_matrix
)

import matplotlib.pyplot as plt
import seaborn as sns
import joblib
import subprocess
from imblearn.over_sampling import SMOTE

print("Libraries loaded")

# ## 2. Run Preprocessing
#
# The shared `preprocess.py` script handles:
# - Dropping `customerID`
# - Fixing `TotalCharges` data type
# - Handling missing values
# - Label encoding categorical columns
# - Standard scaling all features

# In[25]:

subprocess.run(["python", "preprocess.py"])
print("Preprocessing completed")

# ## 3. Load Processed Data

# In[26]:

df = pd.read_csv("../../data/processed/churn_processed.csv")

print("Shape:", df.shape)
df.head()

# ## 4. Separate Features & Target

# In[27]:

X = df.drop("Churn", axis=1)
y = df["Churn"]

print("Feature columns:", list(X.columns))
print("\nTarget distribution:")
print(y.value_counts())

# ## 5. Train / Test Split
#
# > `stratify=y` ensures both train and test sets maintain the same churn ratio.

# In[28]:

```



```

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Train size:", X_train.shape)
print("Test size :", X_test.shape)

# ## 6. Handle Class Imbalance with SMOTE
#
# The Telco dataset is imbalanced churned customers are a minority class.
# **SMOTE** (Synthetic Minority Oversampling Technique) generates synthetic samples for the minority class to
# balance the training set.

# In[29]:

smote = SMOTE(random_state=42)
X_train_sm, y_train_sm = smote.fit_resample(X_train, y_train)

print("After SMOTE:")
print(pd.Series(y_train_sm).value_counts())

# ## 7. Baseline Logistic Regression Model
#
# **Logistic Regression** is a linear classification algorithm that models the probability of a binary
# outcome.
# It uses the sigmoid function to output values between 0 and 1, making it well-suited for binary
# classification tasks like churn prediction.

# In[30]:

lr_model = LogisticRegression(
    max_iter=1000,
    random_state=42,
    solver='lbfgs'
)

lr_model.fit(X_train_sm, y_train_sm)
print("Baseline model training complete!")

# ## 8. Baseline Model Predictions & Evaluation

# In[31]:

y_pred = lr_model.predict(X_test)

print("Baseline Logistic Regression Results:")
print(f" Accuracy   : {accuracy_score(y_test, y_pred):.4f}")
print(f" Precision  : {precision_score(y_test, y_pred):.4f}")
print(f" Recall    : {recall_score(y_test, y_pred):.4f}")
print(f" F1 Score   : {f1_score(y_test, y_pred):.4f}")

# ## 9. Classification Report

# In[32]:

```

```

print(classification_report(y_test, y_pred, target_names=["No Churn", "Churn"]))

# ## 10. Confusion Matrix Baseline

# In[33]:

cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=["No Churn", "Churn"],
            yticklabels=["No Churn", "Churn"])
plt.title("Logistic Regression - Confusion Matrix (Baseline)")
plt.ylabel("Actual")
plt.xlabel("Predicted")
plt.tight_layout()
plt.show()

# ## 11. Cross Validation
#
# 5-fold cross validation is used to verify the model does not overfit on the training set.

# In[34]:

cv_scores = cross_val_score(lr_model, X_train_sm, y_train_sm, cv=5, scoring='accuracy')

print(f"Cross Validation Scores : {cv_scores}")
print(f"Mean CV Accuracy       : {cv_scores.mean():.4f}")
print(f"Standard Deviation      : {cv_scores.std():.4f}")

# ## 12. Hyperparameter Tuning with GridSearchCV
#
# Key hyperparameters tuned:
# - **C** Regularization strength (smaller = stronger regularization)
# - **solver** Optimization algorithm used to fit the model

# In[35]:

param_grid = {
    'C'       : [0.01, 0.1, 1, 10, 100],
    'solver'  : ['lbfgs', 'liblinear'],
    'penalty' : ['l2']
}

grid_search = GridSearchCV(
    LogisticRegression(max_iter=1000, random_state=42),
    param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1
)

grid_search.fit(X_train_sm, y_train_sm)

print("Best Parameters:", grid_search.best_params_)
print("Best CV Score  :", round(grid_search.best_score_, 4))

# ## 13. Tuned Model Predictions & Evaluation

# In[36]:

```

```

best_model = grid_search.best_estimator_
y_pred2 = best_model.predict(X_test)

print("Tuned Logistic Regression Results:")
print(f" Accuracy : {accuracy_score(y_test, y_pred2):.4f}")
print(f" Precision : {precision_score(y_test, y_pred2):.4f}")
print(f" Recall : {recall_score(y_test, y_pred2):.4f}")
print(f" F1 Score : {f1_score(y_test, y_pred2):.4f}")

### 14. Confusion Matrix Tuned Model

# In[38]:

cm2 = confusion_matrix(y_test, y_pred2)

plt.figure(figsize=(6, 4))
sns.heatmap(cm2, annot=True, fmt='d', cmap='Greens',
            xticklabels=["No Churn", "Churn"],
            yticklabels=["No Churn", "Churn"])
plt.title("Logistic Regression - Confusion Matrix (Tuned)")
plt.ylabel("Actual")
plt.xlabel("Predicted")
plt.tight_layout()
plt.show()

### 15. Feature Importance (Coefficients)
#
# In Logistic Regression, the magnitude of each coefficient indicates how strongly that feature influences the
# prediction.
# Positive coefficients increase churn probability; negative coefficients decrease it.

# In[40]:

coef_df = pd.DataFrame({
    'Feature' : X.columns,
    'Coefficient': best_model.coef_[0]
}).sort_values('Coefficient', ascending=False)

plt.figure(figsize=(10, 6))
sns.barplot(data=coef_df, x='Coefficient', y='Feature', palette='coolwarm')
plt.title("Logistic Regression - Feature Coefficients")
plt.tight_layout()
plt.show()

### 16. ROC Curve
#
# The ROC curve shows the trade-off between **True Positive Rate** and **False Positive Rate** at different
# classification thresholds.
# **AUC (Area Under Curve)** closer to 1.0 means better model performance.

# In[42]:

from sklearn.metrics import roc_curve, auc

y_prob = best_model.predict_proba(X_test)[:, 1]

fpr, tpr, _ = roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(6, 4))
plt.plot(fpr, tpr, label=f"AUC = {roc_auc:.2f}", color='blue')

```

```
plt.plot([0, 1], [0, 1], '--', color='gray')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Logistic Regression")
plt.legend()
plt.tight_layout()
plt.show()
```

```
# ## 17. Model Comparison
```

```
#
```

```
# Comparing Baseline vs Tuned Logistic Regression performance.
```

```
# In[43]:
```

```
comparison = pd.DataFrame({
    "Model": ["Logistic Regression", "Tuned Logistic Regression"],
    "Accuracy": [
        accuracy_score(y_test, y_pred),
        accuracy_score(y_test, y_pred2)
    ],
    "F1 Score": [
        f1_score(y_test, y_pred),
        f1_score(y_test, y_pred2)
    ]
})
```

```
comparison
```

```
# ## 18. Save Model
```

```
# In[44]:
```

```
joblib.dump(best_model, "lr_churn_model.pkl")
joblib.dump(X.columns, "lr_feature_columns.pkl")
```

```
print("Model saved successfully")
```

```
# ## 19. Final Summary
```

```
# In[45]:
```

```
print("Logistic Regression Model Completed Successfully")
```

```
--- random_forest.txt ---
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix,
    roc_curve,
```

```

    roc_auc_score
)

import warnings
warnings.filterwarnings('ignore')

# Plot style
sns.set_style('whitegrid')
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['font.size'] = 12

print('Libraries imported successfully!')

# Load the preprocessed dataset
df = pd.read_csv('../data/processed/churn_processed.csv')

print(f'Dataset shape: {df.shape}')
print(f'Number of features: {df.shape[1] - 1}')
print(f'Number of samples: {df.shape[0]}')
df.head()

# Check target distribution
print('Target Distribution:')
print(df['Churn'].value_counts())
print(f'\nChurn rate: {df["Churn"].mean() * 100:.2f}%')

# Separate features and target
X = df.drop('Churn', axis=1)
y = df['Churn']

print(f'Features shape: {X.shape}')
print(f'Target shape: {y.shape}')
print(f'\nFeature columns: {list(X.columns)}')

# Split data into training and testing sets (80/20)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print(f'Training set: {X_train.shape[0]} samples')
print(f'Testing set: {X_test.shape[0]} samples')
print(f'\nTraining churn rate: {y_train.mean() * 100:.2f}%')
print(f'Testing churn rate: {y_test.mean() * 100:.2f}%')

# Train with default parameters first
rf_default = RandomForestClassifier(random_state=42, n_jobs=-1)
rf_default.fit(X_train, y_train)

# Predictions
y_pred_default = rf_default.predict(X_test)

# Evaluate
print('=== Default Random Forest Results ===')
print(f'Accuracy: {accuracy_score(y_test, y_pred_default):.4f}')
print(f'Precision: {precision_score(y_test, y_pred_default):.4f}')
print(f'Recall: {recall_score(y_test, y_pred_default):.4f}')
print(f'F1 Score: {f1_score(y_test, y_pred_default):.4f}')

# Define parameter grid
param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [10, 20, 30, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

# Grid search with cross-validation
rf = RandomForestClassifier(random_state=42, n_jobs=-1)

```

```

grid_search = GridSearchCV(
    estimator=rf,
    param_grid=param_grid,
    cv=5,
    scoring='f1',
    n_jobs=-1,
    verbose=1
)

print('Starting GridSearchCV... (this may take a few minutes)')
grid_search.fit(X_train, y_train)

print(f'\nBest Parameters: {grid_search.best_params_}')
print(f'Best CV F1 Score: {grid_search.best_score_:.4f}')

# Get the best model
best_rf = grid_search.best_estimator_

# Predictions with the best model
y_pred = best_rf.predict(X_test)
y_pred_proba = best_rf.predict_proba(X_test)[:, 1]

# === Core Metrics ===
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
auc_score = roc_auc_score(y_test, y_pred_proba)

print('=' * 45)
print(' TUNED RANDOM FOREST EVALUATION RESULTS')
print('=' * 45)
print(f' Accuracy: {accuracy:.4f} ({accuracy * 100:.2f}%)')
print(f' Precision: {precision:.4f} ({precision * 100:.2f}%)')
print(f' Recall: {recall:.4f} ({recall * 100:.2f}%)')
print(f' F1 Score: {f1:.4f} ({f1 * 100:.2f}%)')
print(f' AUC-ROC: {auc_score:.4f} ({auc_score * 100:.2f}%)')
print('=' * 45)

# Detailed classification report
print('\nDetailed Classification Report:\n')
print(classification_report(y_test, y_pred, target_names=['Not Churn (0)', 'Churn (1)']))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

fig, ax = plt.subplots(figsize=(8, 6))
sns.heatmap(
    cm, annot=True, fmt='d', cmap='Blues',
    xticklabels=['Not Churn', 'Churn'],
    yticklabels=['Not Churn', 'Churn'],
    linewidths=1, linecolor='black',
    annot_kws={'size': 16}
)
ax.set_xlabel('Predicted', fontsize=14)
ax.set_ylabel('Actual', fontsize=14)
ax.set_title('Confusion Matrix Random Forest', fontsize=16, fontweight='bold')
plt.tight_layout()
plt.show()

# ROC Curve
fpr, tpr, thresholds = roc_curve(y_test, y_pred_proba)

fig, ax = plt.subplots(figsize=(8, 6))
ax.plot(fpr, tpr, color='#2196F3', lw=2.5, label=f'Random Forest (AUC = {auc_score:.4f})')
ax.plot([0, 1], [0, 1], color='gray', lw=1.5, linestyle='--', label='Random Guess')
ax.fill_between(fpr, tpr, alpha=0.1, color='#2196F3')

```

```

ax.set_xlabel('False Positive Rate', fontsize=14)
ax.set_ylabel('True Positive Rate', fontsize=14)
ax.set_title('ROC Curve Random Forest', fontsize=16, fontweight='bold')
ax.legend(loc='lower right', fontsize=12)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# Feature Importance (Top 15)
importances = best_rf.feature_importances_
feature_importance = pd.DataFrame({
    'Feature': X.columns,
    'Importance': importances
}).sort_values('Importance', ascending=True)

# Plot top 15 features
top_n = 15
top_features = feature_importance.tail(top_n)

fig, ax = plt.subplots(figsize=(10, 8))
colors = plt.cm.viridis(np.linspace(0.2, 0.9, top_n))
ax.barh(top_features['Feature'], top_features['Importance'], color=colors, edgecolor='black', linewidth=0.5)
ax.set_xlabel('Importance', fontsize=14)
ax.set_ylabel('Feature', fontsize=14)
ax.set_title(f'Top {top_n} Feature Importances Random Forest', fontsize=16, fontweight='bold')
ax.grid(True, axis='x', alpha=0.3)
plt.tight_layout()
plt.show()

# Compare default vs tuned model
metrics_names = ['Accuracy', 'Precision', 'Recall', 'F1 Score']

default_scores = [
    accuracy_score(y_test, y_pred_default),
    precision_score(y_test, y_pred_default),
    recall_score(y_test, y_pred_default),
    f1_score(y_test, y_pred_default)
]

tuned_scores = [accuracy, precision, recall, f1]

x = np.arange(len(metrics_names))
width = 0.35

fig, ax = plt.subplots(figsize=(10, 6))
bars1 = ax.bar(x - width/2, default_scores, width, label='Default RF', color='#FF7043', edgecolor='black',
    linewidth=0.5)
bars2 = ax.bar(x + width/2, tuned_scores, width, label='Tuned RF', color='#42A5F5', edgecolor='black',
    linewidth=0.5)

# Add value labels on bars
for bar in bars1:
    height = bar.get_height()
    ax.annotate(f'{height:.3f}', xy=(bar.get_x() + bar.get_width()/2, height),
        xytext=(0, 5), textcoords='offset points', ha='center', fontsize=10)

for bar in bars2:
    height = bar.get_height()
    ax.annotate(f'{height:.3f}', xy=(bar.get_x() + bar.get_width()/2, height),
        xytext=(0, 5), textcoords='offset points', ha='center', fontsize=10)

ax.set_ylabel('Score', fontsize=14)
ax.set_title('Default vs Tuned Random Forest', fontsize=16, fontweight='bold')
ax.set_xticks(x)
ax.set_xticklabels(metrics_names, fontsize=12)
ax.legend(fontsize=12)
ax.set_ylim(0, 1.15)
ax.grid(True, axis='y', alpha=0.3)

```

```

plt.tight_layout()
plt.show()

# Final summary table
summary = pd.DataFrame({
    'Metric': ['Accuracy', 'Precision', 'Recall', 'F1 Score', 'AUC-ROC'],
    'Default RF': [
        accuracy_score(y_test, y_pred_default),
        precision_score(y_test, y_pred_default),
        recall_score(y_test, y_pred_default),
        f1_score(y_test, y_pred_default),
        roc_auc_score(y_test, rf_default.predict_proba(X_test)[: , 1])
    ],
    'Tuned RF': [accuracy, precision, recall, f1, auc_score]
})

summary['Default RF'] = summary['Default RF'].round(4)
summary['Tuned RF'] = summary['Tuned RF'].round(4)
summary['Improvement'] = (summary['Tuned RF'] - summary['Default RF']).round(4)

print('\n' + '=' * 60)
print('          RANDOM FOREST  FINAL SUMMARY')
print('=' * 60)
print(summary.to_string(index=False))
print('=' * 60)
print(f'\nBest hyperparameters: {grid_search.best_params_}')

```